

课程笔记

LangChain/LangGraph ReAct 差异与 LangSmith 监控

基于公开课程资料整理

2025

ReAct
create react agent
LangChain/LangGraph
langsmith

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 17 分钟

视频链接：<https://www.bilibili.com/video/BV1n4n7znEDN>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1	ReAct 范式回顾	2
1.1	从 CoT 到 ReAct	2
1.2	本章小结	2
2	LangChain 实现 ReAct Agent	2
2.1	Prompt 模板设计	2
2.2	三步骤构建流程	3
2.3	工作机制：基于文本模板的迭代	3
2.4	本章小结	3
3	LangGraph 实现 ReAct Agent	3
3.1	基于图的架构	3
3.2	基于消息列表的管理	3
3.3	本章小结	4
4	LangSmith 监控平台	4
4.1	配置与使用	4
4.2	核心调试方法	4
4.3	本章小结	4
5	LangChain vs LangGraph 对比分析	4
5.1	本章小结	4
6	总结与延伸	5

1 ReAct 范式回顾

1.1 从 CoT 到 ReAct

ReAct 是 Agent 领域最经典的工作之一（姚顺雨团队，2022 年底 / 2023 年初发表于 NeurIPS，引用量已超 4000）。它的演进脉络如下：

1. **CoT (Chain of Thought)**：面向回答问题——“Let’s think step by step”，启发了后续的 Prompt Engineering。
2. **Act Only**：语言模型可以和环境交互，产生动作 (Action)，环境返回观察 (Observation)，但没有显式思考过程。
3. **ReAct = Reasoning + Acting**：在产生动作之前先进行 Reasoning (Thought)，降低模型幻觉，更好地 grounding 到合理的动作。

ReAct 的核心循环

Thought → Action → Observation → Thought → ... → Final Answer

ReAct 启发了后续 Agent 的设计和开发，也可能启发了 GPT API 中 Function Calling 特性的诞生（2023 年 6 月）。

关于 Reasoning Model

现代的 Reasoning Model 在回答问题之前已经学会了思考，不需要再显式地通过 Prompt 告诉它“你要先思考”。后续介绍 Reasoning Model 时会展开讨论其使用规范和标准。

1.2 本章小结

ReAct 将推理 (Reasoning) 与行动 (Acting) 结合，是 Agent 设计的基石。理解其 Thought-Action-Observation 三部曲是理解后续所有 Agent 框架的前提。

2 LangChain 实现 ReAct Agent

2.1 Prompt 模板设计

LangChain 中 `create_react_agent` 使用了一个标准化的 Prompt 模板，其设计强调泛化性和通用性：

Answer the following questions as best you can. You have access to the following tools: {tools}. Use the following format: Question / Thought / Action / Action Input / Observation / ... / Final Answer. Begin! Question: {input} {agent_scratchpad}

模板包含两个动态占位符：

- `tools`：可用的工具列表
- `agent_scratchpad`：交互过程中动态维护的历史记录

2.2 三步骤构建流程

1. 声明语言模型（如 GPT-4.1 Nano）
2. 调用 `create_react_agent` 创建 Agent
3. 将 Agent 放入 `AgentExecutor` 中执行

2.3 工作机制：基于文本模板的迭代

LangChain 的 ReAct 实现有一个重要特征——它不使用多轮消息（`message list`），而是将所有历史信息不断追加到同一个 `human` 消息中：

LangChain ReAct 的历史管理方式

每一轮新的交互都是一次新的 LLM 调用。历史信息（Thought、Action、Observation）被追加到 `agent_scratchpad` 中，作为单条 `human` 消息的一部分发送给模型。这意味着没有多轮对话的 `message` 结构，所有上下文都挤在一个消息里。

2.4 本章小结

LangChain 的 ReAct 实现基于 Prompt 模板 + 文本追加的方式管理交互历史，结构简单但缺乏多轮消息的清晰组织。

3 LangGraph 实现 ReAct Agent

3.1 基于图的架构

LangGraph 的 `create_react_agent` 实现更为现代化。只需定义模型和工具即可：

理解 LangGraph 的关键：虚边（条件边）

LangGraph 的 ReAct Agent 包含两个核心节点：

- **Agent 节点**：负责产生工具调用请求
- **Tools 节点**：负责执行工具并返回结果

Agent 节点上有一条**条件边**，决策是否要停止调用工具直接输出答案，还是继续调工具。

3.2 基于消息列表的管理

与 LangChain 不同，LangGraph 完全基于**多轮对话消息**的方式管理交互：

消息流：Human Message → AI Message（含 `tool_calls`）→ Tool Message(s) → AI Message（最终回答）

并行工具调用

LangGraph 支持**并行工具调用**——AI Message 中的 `tool_calls` 是一个列表，可以同时调用多个工具。例如查询“澳网冠军的家乡”时，模型可能并行调用两次搜索工具。

3.3 本章小结

LangGraph 的 ReAct 实现基于图结构和多轮消息列表，架构更清晰，支持并行工具调用。

4 LangSmith 监控平台

4.1 配置与使用

LangSmith 是一个在线监控平台，用于追踪和调试 Agent 的工作流程。配置方式：

1. 在 `.env` 中配置 LangSmith API Key
2. 开启 tracing 功能（设置为 true）
3. 设置 project 名称

4.2 核心调试方法

Debug 的核心 Tips

通过检查模型的**输入输出**来理解其工作流程。重点关注每一次语言模型调用（橙色部分）的输入和输出——搞清楚输入输出，就搞清楚了整个工作流程。所有前续步骤都是为了构建这样一个消息列表。

4.3 本章小结

LangSmith 提供了可视化的 Agent 调试能力，核心方法是通过监控 LLM 调用的输入输出来理解整个工作流程。

5 LangChain vs LangGraph 对比分析

维度	LangChain	LangGraph
历史管理	文本追加到单条消息	多轮消息列表
消息结构	单条 human 消息（含 scratchpad）	Human / AI / Tool 消息序列
工具调用	串行（一次一个）	支持并行（tool_calls 列表）
架构	Prompt 模板 + Executor	Graph（节点 + 条件边）
可视化	需手动追踪	可直接导出 Graph 图

表 1: LangChain 与 LangGraph 的 ReAct 实现对比

5.1 本章小结

LangGraph 在架构清晰度、消息管理和并行能力上全面优于 LangChain 的旧版实现。LangChain 的方式更适理解 ReAct 的原始概念，而 LangGraph 更适合生产环境。

6 总结与延伸

1. **重视基础概念**：越基础的东西越本质，后续所有现代的、强大的 Agent 开发理念都脱胎于经典工作。
2. **理解实现差异**：LangChain 通过模板管理和文本追加，LangGraph 通过多轮消息和图结构——底层机制不同，理解差异有助于选择合适的工具。
3. **善用监控工具**：LangSmith 等平台是理解 Agent 内部工作流程的利器，核心是检查每次 LLM 调用的输入输出。